

Do Transformers Need Three Projections: A Systematic Study of QKV Projection-Sharing Variants

Ali Kayyam • Anusha Madan Gopal • M Anthony Lewis

Problem Statement

Every attention head learns **three** separate linear projections of each token — **Query, Key, Value**. This redundancy is rarely questioned.

CNNs, RNNs and State-Space Models use unified internal representations — Transformers keep three. Is the tripartite design actually necessary?

THREE PROJECTION-SHARING CONSTRAINTS

Q=K=V Tie Query & Key → symmetric attention map

Q-K=V Tie Key & Value → caches K only (50% cache)

Q=K=V Single projection → most constrained

These architectures enable **practical inference on memory-constrained edge / on-device hardware**

Method: Projection Sharing

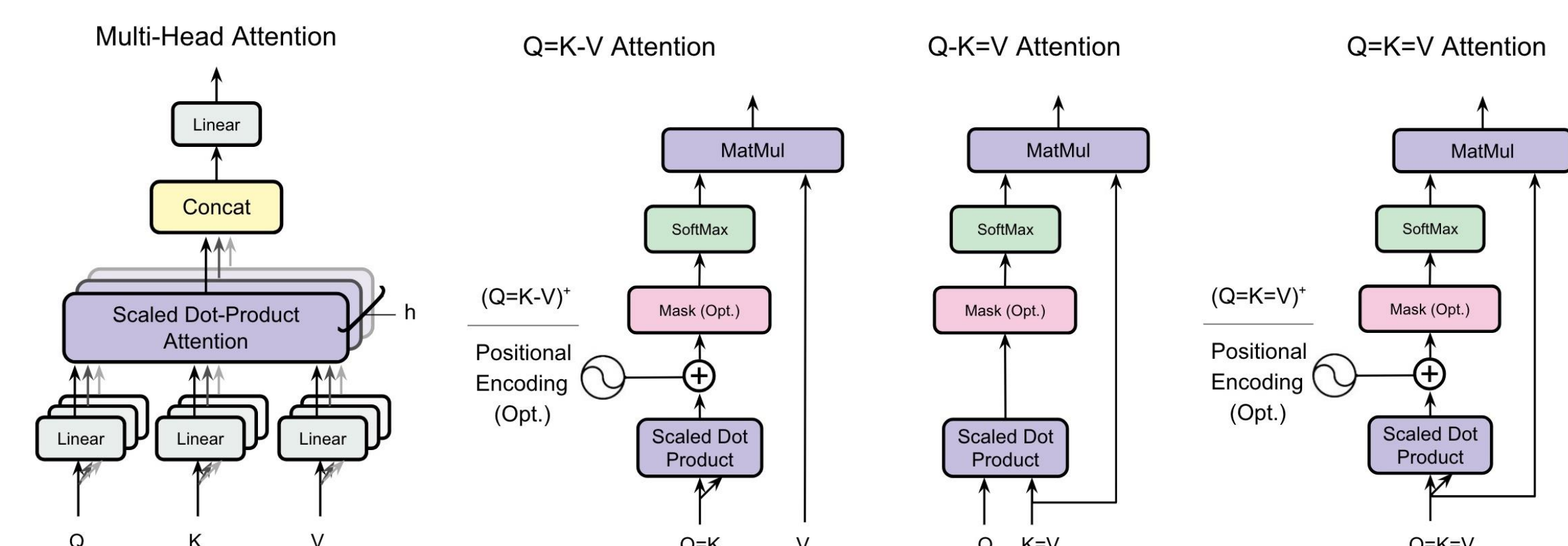


Fig: Attention variants — same core mechanism, only the tied projections change.

Projection cost drops 33%: $QKV \ 3nd^2 \rightarrow 2nd^2 \rightarrow nd^2$

(X)+ variants add a fixed 2D positional encoding to restore directional asymmetry on non-causal tasks (vision, synthetic).

Main Results: Language Modeling

300M and 1.2B parameter models · 10B tokens · SlimPajama. Q-K=V is the clear winner among projection

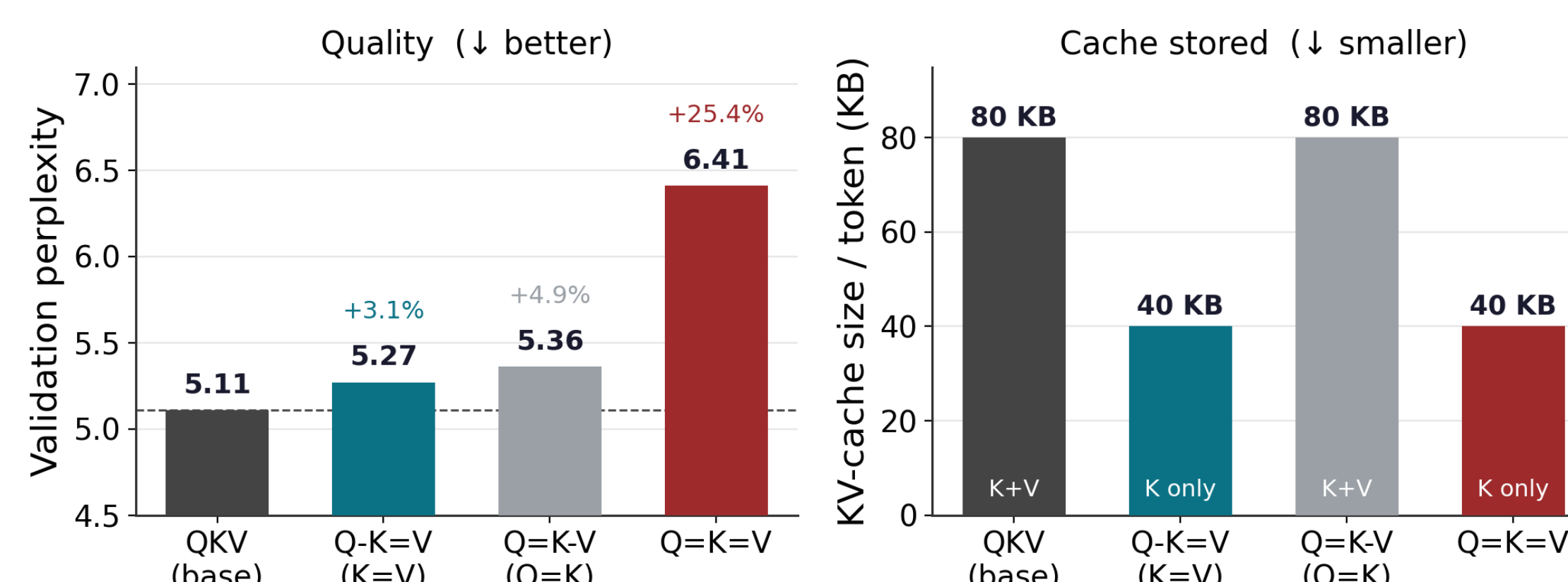


Fig: Validation perplexity (left) and KV-cache size per token (right).

+3.1%
perplexity cost

50%
cache reduction

+25.4%
Q=K=V collapses

Setup Task: GPT-style autoregressive LM · Data: SlimPajama, ~10B tokens · Models: 300M (20L, d=1024, 16h) & 1.2B (22L, d=2048, 32h), vocab 50,304, seq 2048 · AdamW, bf16, 8×A100, DDP · Eval: val PPL + 5-shot lm-eval-harness · Only the attention projection differs.

Results (val perplexity) QKV 5.11 · Q-K=V 5.27 (+3.1%) · Q=K=V 5.36 (+4.9%, no cache benefit)

Why It Works?

✓ Q-K=V preserves quality

Keys & Values can share representational space while **attention QK^T stays asymmetric**. Trained QKV models confirm it: $\cos(K,V)=0.73$, effective rank 687 vs 702/1024; Q stays distinct: $\cos(Q,K)=0.42$, $\cos(Q,V)=0.31$.

✗ Q=K=V breaks directionality

Forcing $Q = K$ makes the attention map **symmetric (KK^T)**, breaking the causal directionality language modeling needs: +4.9% perplexity with zero cache benefit. Q=K=V inherits both flaws.

Key Takeaways

- K = V is enough**
50% KV-cache cut at only +3.1% (300M) / +2.48% (1.2B) perplexity.
- Asymmetry is what matters**
Quality needs Q distinct from a shared K-V — not three projections.
- Orthogonal to head sharing**
Stacks with GQA/MQA for up to 96.9% cache reduction.

Stacking with Head Sharing

Projection sharing ($K = V$) is orthogonal to head sharing (GQA / MQA) — the two stack multiplicatively.

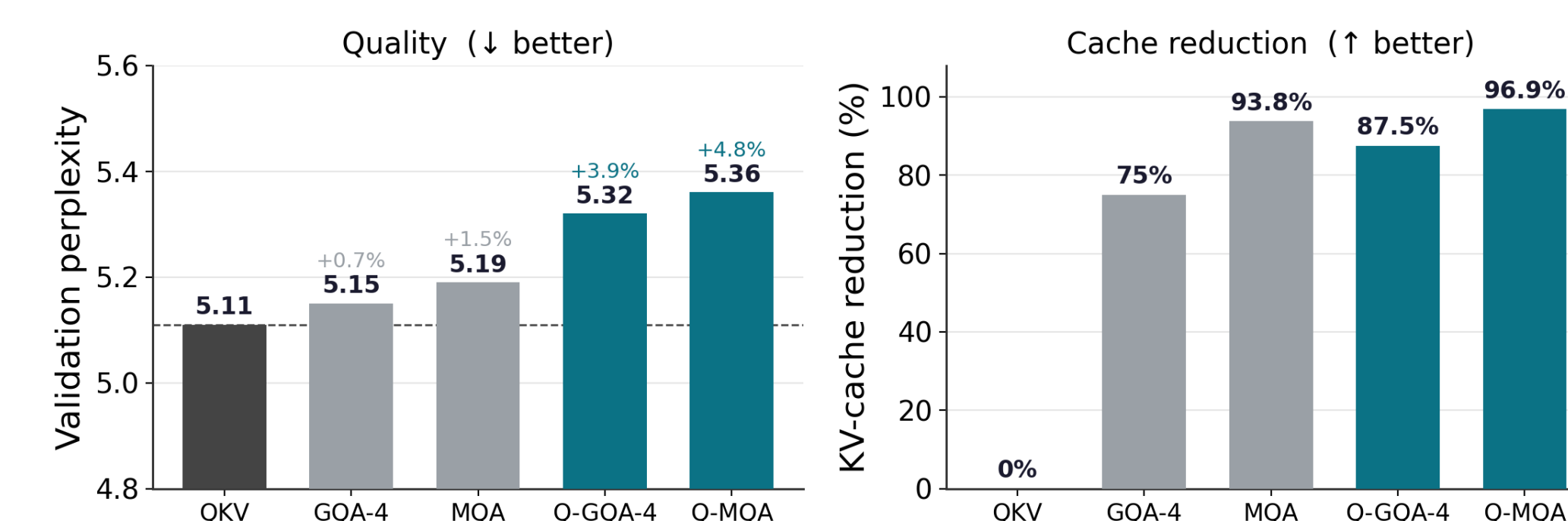


Fig: Head sharing and combined projection + head sharing (perplexity & cache reduction).

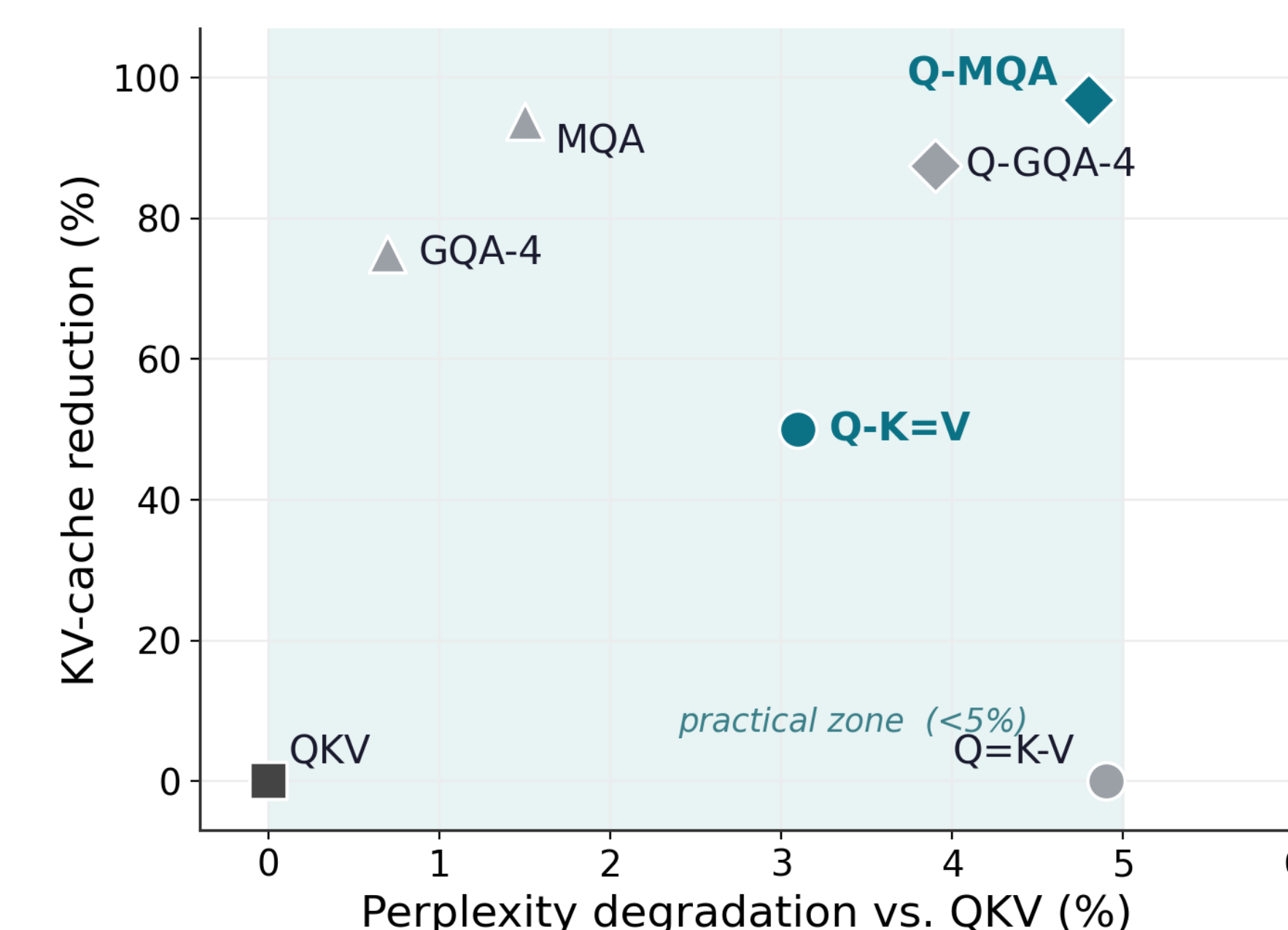


Fig: Efficiency–quality Pareto frontier. Q-K=V fills the gap to head-sharing methods.

Deployment & Impact

 Generous budget	GQA-4	75% cache ↓ +0.7% ppl
 Moderate budget	Q-K=V	50% cache ↓ +3.1% ppl
 Tight budget	Q-GQA-4	87.5% cache ↓ +3.9% ppl
 Very tight budget	Q-MQA	96.9% cache ↓ +4.8% ppl

Perplexity ≠ capability. At 1.2B, Q-K=V trails QKV by 2.48% perplexity but only **0.41% downstream accuracy** — while halving the KV cache. Up to 67× cache reduction with Q-MQA enables practical billion-parameter edge inference.

Conclusion: Transformers don't need three projections — tying $K = V$ halves the KV cache at minimal quality cost and stacks with head sharing.

Code: [Brainchip-Inc/Do-Transformers-Need-3-Projections](https://github.com/Brainchip-Inc/Do-Transformers-Need-3-Projections)